NEIL GANDHI | TECHNICAL WRITEUP | PERSONAL PROJECT

Music Player

A slowed-and-reverb music player with a 310-track library, streaming from object storage and running with no server and no database.

Live Site: neilg.ca

1. Executive Summary

Most music players are built around a catalogue someone else owns. I built this one around my own library — slowed-and-reverb versions of songs I already listen to, 310 tracks as of November 2026, streamed from a Cloudflare R2 bucket I control. The player lives in the Music view of my personal site, neilg.ca, inside the same single self-contained index.html as the rest of the site. There is no backend, no database, and no API key in the page: the audio comes from R2, the metadata comes from a Google Sheet I fill in, and the playlists live in the browser's localStorage.

The hard part is not playback. Play, pause, and seek are solved problems. The hard part is the library: 761 MP3s migrated from an archive.org backup, a metadata sheet that decides what appears where, and a set of rules for artists, albums, and removals that have to hold for every track or the library falls apart. It is also a design exercise: one phone screen, no scroll, dark mode only, and a visualizer that reacts to the actual audio through the Web Audio API rather than a canned animation.

This writeup is written for engineers and hiring managers evaluating the project: what it does, how the audio and metadata flow, the three problems that were genuinely hard, and how to set it up yourself.

The short version:

audio in R2, metadata in a sheet, one static page on top — with the library curated by hand, not by an algorithm.

2. Architecture & System Design

There is no application server behind this player, and that is a deliberate choice, not a missing piece. The system is a static page plus two storage services: Cloudflare R2 holds the audio, a Google Sheet holds the metadata, and the browser holds the playlists. Hosting stays a plain GitHub Pages site. If R2 is down, nothing plays; if the Sheet is wrong, the library is wrong; but there is no server to crash and no database to keep in sync.

Google Sheet	R2 bucket neil-music	index.html	Browser localStorage
metadata sidecar	659 objects, public URLs	one page, phone frame	playlists + picks

Audio: R2 public base URL + URL-encoded filename. Metadata: the Sheet, mirrored to a local CSV. No request leaves the browser except the audio fetch.

How a track gets from the sheet to the speaker

1. **I add a row to the Sheet.** Filename, artist, album, title, effect, track number. The filename is the key — stream URLs are built from it, so a kept track's filename is never renamed.
2. **The file goes into R2.** The MP3 lives in the neil-music bucket under its filename; the public base URL plus the URL-encoded filename is the stream URL. CORS allows GET and HEAD from any origin, so the browser can fetch it directly.
3. **The page carries the library.** The player reads its track list from the page itself — artist, album, title, and the R2 URL for each track — and renders Artist → album → song from it. The Sheet is where I curate; the page is what plays.
4. **Playback runs entirely in the browser.** An <audio> element streams from R2; an AnalyserNode taps the same stream for the visualizers; playlists and picks are read from and written to localStorage. No request leaves the browser except the audio fetch.

Why this stack

R2, because audio files are large, static, and read-heavy — object storage with a public URL is the cheapest honest way to serve them, and there is no reason to proxy them through a server I would have to maintain. A Google Sheet, because the metadata is a table I edit by hand a few rows at a time, and a spreadsheet is the right tool for that; a database would add a backend for a job a sheet already does. localStorage, because playlists are personal and per-device — they belong in the browser, not on a server. One self-contained page, because the player is a front-end problem; a build step would add a toolchain for a file that is already a single HTML document.

The metadata layer is one sheet, seven columns

Column	What it holds
filename	The MP3 filename in R2 — the stream key, never renamed once a track is kept
artist	Main artist only; featured artists go in the title as "ft. X"
album	Album name; "single" becomes Singles, "Leaks" becomes Leaks, "remove" drops the track
title	Bare title, no " (S + R)"; the page appends it at render
effect	The effect applied to the track (slowed + reverb)
needs_check	Flag for tracks that need a listen before I keep them
track_no	Track number within the album

The library came from a migration, not from uploads one at a time. On November 3, 2026 I moved 761 MP3s from an archive.org item, music-folder1, into R2; the archive.org item stays as the offline backup. A cleanup on November 7 deleted 102 bass-boosted files; 659 objects remain in the bucket and zero files match "bass".

Tech stack

Layer	Choice
Audio storage / delivery	Cloudflare R2 bucket neil-music; public base https://pub-310c4716cfa349e883e267dd989c95a3.r2.dev/; CORS open for GET/HEAD
Metadata	Google Sheet "Slowed tracks - metadata" (ID 1TXWWBgSEkZZx42WyvkkvuumoFjQS-VtTfinBeXOqLNw); local CSV mirror, 330 data rows
Hosting	GitHub Pages, repo Quert-Z.github.io, domain neilg.ca
Frontend	One self-contained index.html (HTML, CSS, vanilla JavaScript) in a phone frame; dark mode only; no build step
Playlists / picks	localStorage, key music-player.user-playlists.v1

On open, the splash shows a robot mascot, "My Slowed & Reverb Collection", and a Headphone Advisory; after Continue, a welcome tour walks through Albums, Neil's Picks, and All Songs, one short screen each, skippable throughout. The Albums grid is A–Z with Singles pinned and Leaks at the bottom.

3. Core Features & Implementation

Challenge 1: a library that stays consistent across 310 tracks

The Sheet looks simple until two artists share an album. The rules below are the whole consistency model — every track passes through them, and there is no manual fix-up step after. Combo credits are split on "&" or "," so each artist gets their own tile and a shared album appears under EACH artist. JACKBOYS is the one exception: it is never split, and its album is not shared with Travis Scott. Album routing is by value: "single" becomes a per-artist Singles album, "Leaks" becomes a per-artist Leaks album, and "remove" drops the track; "remove this" or "delete this" appended to the filename cell does the same. Titles are stored bare and rendered with " (S + R)" appended, under the subtitle "Mixed & Prod by Neil".

```
# every track passes through these rules - no manual fix-up after
artists = split_credit(sheet.artist)      # on "&" or ","
if artist == "JACKBOYS":                  # never split, never shared
    artists = ["JACKBOYS"]
for a in artists:
    place(album, under=a)

album == "single"  -> per-artist "Singles"
album == "Leaks"   -> per-artist "Leaks"
album == "remove"  -> drop the track
"remove this" / "delete this" in filename -> drop the track
title stored bare -> rendered as title + " (S + R)"
```

The filename rule is the one that protects the whole system. Stream URLs are built from filenames, so renaming a kept track would break its stream and orphan its row in the Sheet. That

is why the cleanup deleted files by name rather than renaming anything, and why the one row with a blank filename is a decision, not a typo to quietly fix.

Challenge 2: playback that feels instant without preloading

Preloading was deliberately rejected. With 310 tracks, preloading burns bandwidth and memory for songs the listener may never play, and on a phone that cost is real. So the player starts streaming on tap and has to feel fast anyway. The queue is the mechanism that makes that work: a mini-bar queue button sits first in the transport row, play/pause last, so the queue control is where the thumb already is. Tapping it queues the current song; a queue sheet shows what is up next with per-row remove and Clear, a badge counts the queue, and every song row has its own queue button. The transport row also carries like, shuffle, repeat, and speed pills.

Neil's Picks is the curated side of the same idea: a read-only section whose order I set (create and add were removed), stored in localStorage alongside custom playlists, which the listener can create, fill, rename, and delete. The distinction is deliberate — picks are mine, playlists are theirs — and both live in the browser, not on a server.

All Songs sorts by Title A–Z by default or Artist A–Z; album and pick screens stay in playlist order, because an album is a sequence, not a sort key.

Challenge 3: a visualizer that listens to the music

A fake visualizer is easy — animate bars on a sine wave and nobody can prove otherwise at a glance. This one taps the actual audio. An AnalyserNode sits between the audio element and the speakers, and both the Playing Now visualizer and the 44px header visualizer under the "320kbps" label read from it:

```
var srcNode = vizAC.createMediaElementSource(audio);
vizAnalyser = vizAC.createAnalyser();
vizAnalyser.fftSize = 2048;
vizAnalyser.smoothingTimeConstant = 0.82;
srcNode.connect(vizAnalyser);
vizAnalyser.connect(vizAC.destination);
```

The header visualizer is the same node with a different mapping: 12 bars spread across the lower 45% of the frequency bins, sampled denser at the low end, where the kick and bass live — so the bars move with the beat rather than with the clock:

```
var HV_BARS = 12;
// per bar: sample the lower 45% of bins, denser at the low end
var frac = i / HV_BARS;
var bin = Math.floor(Math.pow(frac, 1.5) * hvFreq.length * 0.45) + 1;
target = hvFreq[bin] / 255;
```

When nothing is playing, the bars idle on a sine animation instead of going flat, and under prefers-reduced-motion they hold steady — the visualizer is decoration when there is no audio, and it should not pretend otherwise. The standalone visualizer pages use the same node with a different effect: a letterboxed cinema frame, title card, big play button, and a bottom bar (play, seek, speed 0.80–1.25x, volume, fullscreen) over film grain, with a bass/kick-reactive animation — fast-attack/slow-release halo, core pulse, anamorphic flare, and clean beat rings.

4. Setup & Usage Guide

Prerequisites

- A Cloudflare account (free) with an R2 bucket; the bucket needs a public URL and CORS allowing GET and HEAD.
- A Google Sheet with the seven metadata columns, plus a local CSV mirror.
- A GitHub repository with Pages enabled, holding index.html.
- No API keys, no backend, no database. The page streams audio straight from R2.

Set up the storage

Create the R2 bucket, enable public access, and set CORS to allow origins "*" for GET and HEAD. Upload the MP3s. Filenames are the stream keys — decide them before uploading, because renaming a kept track breaks its URL. Create the Sheet with columns filename | artist | album | title | effect | needs_check | track_no, sorted A–Z by filename, and keep a local CSV mirror alongside it.

Put the player on the page

```
git clone https://github.com/Quert-Z/Quert-Z.github.io.git
cd Quert-Z.github.io
```

The player is the Music view inside index.html; the track list lives in the page, and each track's audio URL is the R2 public base plus its URL-encoded filename. Push index.html to deploy. One note from experience: an index.html taken from the site's Deploy-tab download must go through the portable patch script before pushing (the download bundles a runtime shim that hangs on GitHub Pages), and Pages needs roughly five minutes to deploy before you check.

Using the player

Open neilg.ca and enter the Music view. The splash shows the robot mascot, "My Slowed & Reverb Collection", and the Headphone Advisory; tap Continue, then the welcome tour walks through Albums, Neil's Picks, and All Songs (Skip tour any time). Browse Artist → album → song, or open All Songs (Title A–Z by default, or Artist A–Z). Tap a song to open Playing Now. Queue with the queue button in the transport row or the per-row queue button on any song; open the queue sheet to remove a row or Clear. Like, shuffle, repeat, and speed are in the transport row. Neil's Picks is the curated read-only section; custom playlists (create, pick songs, rename, delete) are stored in localStorage on your device.